

Exercise Set 2: Operator Overloading

Category 1: Arithmetic & Logic Operators

Exercise 1 Easy

Create a class `Fraction` with numerator and denominator. Overload the `+` operator to add two fractions. The result should be in simplified form.

Expected Implementation Details:

Store numerator and denominator as private integers. Overload as member function:

`Fraction operator+(const Fraction& other) const`. Addition formula: $(a/b) + (c/d) = (ad + bc)/(bd)$. Include private `gcd(int a, int b)` function using Euclidean algorithm. Simplify result by dividing both numerator and denominator by their GCD. Handle negative fractions correctly. Constructor should prevent zero denominator. Test: $1/2 + 1/3 = 5/6$, $2/4 + 1/4 = 3/4$.

Exercise 2 Easy

Design a class `Complex` representing complex numbers. Overload `+`, `-`, `*` operators for complex number arithmetic.

Expected Implementation Details:

Private members: `double real, imag`. Implement as member functions returning `Complex` objects. Addition: $(a+bi) + (c+di) = (a+c) + (b+d)i$. Subtraction: $(a+bi) - (c+di) = (a-c) + (b-d)i$. Multiplication: $(a+bi) \times (c+di) = (ac-bd) + (ad+bc)i$. Include `magnitude()` returning $\sqrt{\text{real}^2 + \text{imag}^2}$. Add display method showing "a + bi" format (handle negative imaginary part). Test with $(3+4i) \times (1-2i) = 11-2i$.

Exercise 3

Easy

Create a class `Time` (hours, minutes, seconds). Overload the `+` operator to add two time durations. Handle overflow properly (e.g., 70 seconds = 1 minute 10 seconds).

Expected Implementation Details:

Store as three integers. `operator+` should add corresponding fields then normalize: if seconds ≥ 60 , add to minutes and reduce seconds; if minutes ≥ 60 , add to hours and reduce minutes. Consider whether to limit hours to 24 or allow unlimited duration. Include `toSeconds()` helper converting to total seconds for easier arithmetic. Display as "HH:MM:SS" with leading zeros. Test: 1:45:30 + 2:20:45 = 4:06:15.

Exercise 4

Intermediate

Implement a class `Matrix2x2` and overload `+`, `-`, `*` operators for matrix operations. Include scalar multiplication as well.

Expected Implementation Details:

Store as `double m[2][2]`. Matrix addition/subtraction: element-wise operations. Matrix multiplication: $\text{result}[i][j] = \sum (a[i][k] \times b[k][j])$. For scalar multiplication, overload `operator*(double scalar)` as member and `operator*(double, const Matrix2x2&)` as friend for commutativity. Include `determinant()` and `transpose()` methods. Test matrix multiplication is not commutative: $AB \neq BA$ in general.

Exercise 5

Easy

Create a class `Distance` with feet and inches. Overload `+` and `-` operators. Ensure proper conversion (12 inches = 1 foot).

Expected Implementation Details:

Store feet and inches as integers. In operators, first convert to total inches, perform operation, then convert back. Normalize function: inches = total % 12, feet = total / 12. Handle negative distances appropriately (decide if allowed). For subtraction, consider what happens when result would be negative. Include comparison with zero. Test: 5'11" + 0'2" = 6'1", 3'4" - 1'8" = 1'8".

Exercise 6

Intermediate

Design a class `Set` representing a mathematical set. Overload `|` for union, `&` for intersection, and `-` for difference operations.

Expected Implementation Details:

Use vector or array to store unique integers. Maintain uniqueness in all operations. Union (`|`): combine elements from both sets, remove duplicates. Intersection (`&`): keep only common elements. Difference (`-`): elements in first but not in second. Include `contains(int)`, `add(int)`, `size()`. Consider using sorted storage for efficiency. Implement `operator==` for set equality. Test: $\{1,2,3\} \cup \{3,4,5\} = \{1,2,3,4,5\}$.

Exercise 7

Easy

Implement a class `Currency` with rupees and paise. Overload arithmetic operators `+`, `-`, `*` (with scalar), and `/` (with scalar).

Expected Implementation Details:

Store rupees and paise as integers (paise: 0-99). Convert to total paise for calculations, then normalize. For scalar multiplication/division, handle rounding appropriately. Prevent division by zero. Include `operator*(int)` and friend `operator*(int, Currency)` for commutativity. Display as "₹RR.PP" format. Consider overflow for large amounts. Test: ₹10.50 * 3 = ₹31.50.

Exercise 8

Difficult

Create a class `Polynomial` representing polynomials. Overload `+`, `-`, and `*` operators. Store coefficients efficiently and handle polynomials of different degrees.

Expected Implementation Details:

Store coefficients in dynamic array or vector, index represents power. For addition/subtraction, align coefficients by degree. Multiplication: use nested loops, coefficient of $x^k = \sum (a[i] \times b[j])$ where $i+j=k$. Trim leading zeros after operations. Include `degree()`, `evaluate(double x)`, and string representation. Handle zero polynomial specially. Test: $(x^2 + 2x + 1) \times (x + 1) = x^3 + 3x^2 + 3x + 1$.

Exercise 9

Intermediate

Design a class `BitVector` for bit manipulation. Overload `&`, `|`, `^`, `~`, `<<`, and `>>` operators for bitwise operations.

Expected Implementation Details:

Store bits in array of unsigned integers (32 or 64 bits each). Calculate array size based on number of bits needed. Implement element-wise operations for `&`, `|`, `^`. For `~`, flip all bits up to vector size. Shift operators need special handling across array boundaries. Include `set(index)`, `clear(index)`, `test(index)`. Handle different sized `BitVectors` in binary operations. Test all operations with boundary cases.

Exercise 10

Easy

Create a class `Angle` storing degrees. Overload `+` and `-` operators. Ensure the result is always normalized (0-359 degrees).

Expected Implementation Details:

Store angle as double. After any operation, normalize: `while(angle >= 360) angle -= 360;` `while(angle < 0) angle += 360;` Or use modulo: `angle = fmod(angle, 360); if(angle < 0) angle += 360;` Include conversion methods `toRadians()` and static `fromRadians()`. Display with degree symbol. Test: $350^\circ + 20^\circ = 10^\circ$, $30^\circ - 60^\circ = 330^\circ$.

Category 2: Comparison Operators

Exercise 11

Easy

For the `Fraction` class, overload all comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) to compare fractions.

Expected Implementation Details:

For equality, compare simplified fractions or cross-multiply: $a/b == c/d$ if $ad == bc$. For ordering, convert to common denominator or use: $a/b < c/d$ if $ad < bc$ (assuming positive denominators). Implement `operator<` and `operator==` first, derive others: `!=` is `!(==)`, `<=` is `(< || ==)`, `>` is `!(<=)`, `>=` is `!(<)`. Handle negative fractions correctly in comparisons. Test: $1/2 < 2/3$, $2/4 == 1/2$.

Exercise 12

Easy

Create a class `Date` with day, month, year. Overload all comparison operators to compare dates chronologically.

Expected Implementation Details:

Compare year first, then month, then day. Can convert to days since epoch for easier comparison. Implement `operator<` with cascading comparisons: if years differ, return year comparison; if months differ, return month comparison; return day comparison. Use this to implement others. Consider leap years if relevant. Include `daysSince(const Date& other)` method. Test: `01/03/2024 > 28/02/2024`.

Exercise 13

Intermediate

Design a class `Version` (e.g., "2.4.1"). Overload comparison operators to compare software versions correctly (2.10.0 > 2.9.0).

Expected Implementation Details:

Store major, minor, patch as integers. Parse string in constructor using `find()` and `substr()`. Compare major first, then minor, then patch. Critical: 2.10 > 2.9 (not string comparison!). Handle missing parts: "2.1" equals "2.1.0". Implement `toString()` for display. Consider pre-release tags (2.0.0-alpha < 2.0.0). Test edge cases: 1.0.0 vs 1.0, 2.10.1 vs 2.9.9.

Exercise 14

Easy

Implement a class `Student` with name and marks. Overload comparison operators to compare students based on marks, with name as tiebreaker.

Expected Implementation Details:

Primary comparison on marks (higher is "greater"), secondary on name (alphabetical). In `operator<`: if marks differ, return marks comparison; else return `name < other.name`. This creates consistent ordering for sorting. Include `getRank()` method. Consider case-insensitive name comparison. Test with equal marks but different names, ensure transitivity: if `a < b` and `b < c`, then `a < c`.

Exercise 15

Intermediate

Create a class `IPv4Address`. Overload comparison operators to compare IP addresses correctly ($192.168.1.100 > 192.168.1.99$).

Expected Implementation Details:

Store four octets as unsigned char or int. Compare octet by octet from left to right. Can also convert to 32-bit unsigned integer for easier comparison: $(\text{octets}[0] \ll 24) | (\text{octets}[1] \ll 16) | (\text{octets}[2] \ll 8) | \text{octets}[3]$. Include validation in constructor. Handle special addresses in display. Test: $10.0.0.1 < 192.168.0.1$, $192.168.1.100 > 192.168.1.99$.

Exercise 16

Difficult

Design a class `BigInteger` for arbitrary precision integers. Overload all comparison operators to handle numbers larger than built-in types.

Expected Implementation Details:

Store digits in vector (least significant first for easier arithmetic). Handle sign separately. For comparison: first compare signs, then lengths (for same sign), then digit-by-digit from most significant. Implement careful logic for negative numbers. Include string constructor and `toString()`. Handle leading zeros. Test with numbers exceeding long long range. Consider: $-999 < -99 < 0 < 99 < 999$.

Category 3: Data Access Operators

Exercise 17

Easy

Create a class `Array` wrapping a dynamic array. Overload the `[]` operator for element access with bounds checking.

Expected Implementation Details:

Return reference: `int& operator[](int index)` for modification, and `const int& operator[](int index) const` for const objects. Check: `if(index < 0 || index >= size)` throw `out_of_range` exception or handle error. Store size and pointer to dynamic array. Include `at(index)` method that always checks bounds. Destructor must delete[] array. Test both reading and writing through `operator[]`.

Exercise 18

Intermediate

Implement a class `Matrix` (2D). Overload `[]` operator to allow syntax like `matrix[i][j]`. Hint: Return a proxy object.

Expected Implementation Details:

First `[]` returns a proxy object representing a row. Proxy class has its own `operator[]` for column access. Example: `class Proxy { int* row; int cols; operator[](int j) { return row[j]; };` `Matrix::operator[]` returns `Proxy(data[i], cols)`. Include bounds checking in both operators. Alternative: use `operator()` with two parameters: `matrix(i,j)`. Test chained access and assignment.

Exercise 19

Easy

Design a class `SmartPointer` that wraps a raw pointer. Overload `*` (dereference) and `->` (arrow) operators.

Expected Implementation Details:

`T& operator*()` returns reference to pointed object. `T* operator->()` returns the pointer for member access. Check for null before dereferencing. Include `get()` returning raw pointer, `reset(T* p)` to change pointer. Basic implementation without reference counting. Destructor deletes the pointer. Prevent copying or implement proper copy semantics. Test: `SmartPointer sp(new Student); sp->getName();`

Exercise 20

Difficult

Create a class `FunctionWrapper` that stores a function pointer. Overload the `()` operator to make objects callable with various argument counts.

Expected Implementation Details:

Template class storing function pointer. Overload `operator()` with different parameter counts or use variadic templates. Example for single parameter: `template class FunctionWrapper { R (*fn)(A); R operator()(A arg) { return fn(arg); } };`. Include null check. For multiple overloads, need multiple `operator()` definitions. Test with different function signatures.

Exercise 21

Intermediate

Implement a class `SafeArray` with negative indexing support. Overload `[]` to accept negative indices (e.g., -1 for last element).

Expected Implementation Details:

Convert negative indices: if(`index < 0`) `index = size + index`. So -1 becomes `size-1` (last element). Still check bounds after conversion. Support both positive and negative bounds checking. Include `front()` and `back()` convenience methods. Consider circular indexing as extension. Test: `array[-1]` accesses last element, `array[-2]` second-to-last.

Exercise 22

Easy

Design a class `ConfigMap` storing key-value pairs. Overload `[]` operator to access values by string keys.

Expected Implementation Details:

Use `map` or implement with parallel arrays/vectors. `string& operator[](const string& key)` returns reference to value. If key doesn't exist, create with default value (like `std::map`). Include `contains(key)`, `remove(key)` methods. Consider `const` version that doesn't create missing keys. Handle case sensitivity. Test: `config["host"] = "localhost"; string h = config["host"]`.

Exercise 23

Intermediate

Create an iterator class `ListIterator` for a custom linked list. Overload `*`, `->`, and `[]` operators appropriately.

Expected Implementation Details:

Store pointer to current node. `operator*` returns node's data. `operator->` returns pointer to data. For random access `operator[]`, traverse n nodes from current position (inefficient but possible). Include `operator++` for iteration. Check for null/end. Make list class's `begin()` and `end()` return iterators. Test complete iteration and access patterns.

Exercise 24

Difficult

Implement a class `Tensor3D` for 3D data. Overload `[]` to allow chained access like `tensor[i][j][k]` with proper bounds checking.

Expected Implementation Details:

Need two proxy classes: first `[]` returns `Slice2D` proxy, its `[]` returns `Slice1D` proxy, its `[]` returns actual element. Store data as 1D array with index calculation: `data[i*(dim2*dim3) + j*dim3 + k]`. Each proxy stores necessary dimensions and offset. Implement bounds checking at each level. Include `reshape()` method. Test all three dimensions and verify correct element access.

Category 4: Increment/Decrement Operators

Exercise 25

Easy

For the `Date` class, overload pre and post increment/decrement operators to move to next/previous day.

Expected Implementation Details:

Pre-increment: `Date& operator++()` modifies and returns `*this`. Post-increment: `Date operator++(int)` saves old value, modifies, returns old. Handle month/year boundaries: incrementing Dec 31 gives Jan 1 of next year. Consider leap years for February. Helper function for days in month. Decrement similarly handles boundaries. Test month and year transitions.

Exercise 26

Easy

Create a class `Counter` with upper and lower limits. Overload `++` and `--` (both versions) with wraparound behavior.

Expected Implementation Details:

Store value, min, max. When incrementing at max, wrap to min. When decrementing at min, wrap to max. Pre-increment: `if(++value > max) value = min; return *this;` Post versions save old value first. Include `setValue()` with bounds checking. Consider making limits const members. Test wraparound in both directions.

Exercise 27

Intermediate

Design a class `Iterator` for a custom container. Overload pre/post increment to traverse elements, and pre/post decrement for reverse traversal.

Expected Implementation Details:

Store pointer to current element and container. Pre-increment advances pointer and returns `*this`. Post-increment returns copy with old position. Check bounds: don't go past `end()` or before `begin()`. Include comparison operators for iterator equality. Make container's `begin()` and `end()` return iterators. Support both forward and reverse iteration. Test complete traversal in both directions.

Exercise 28

Easy

Implement a class `TimeStamp` with seconds precision. Overload increment/decrement operators to add/subtract one second with proper overflow handling.

Expected Implementation Details:

Store hours, minutes, seconds. Increment: `if(++seconds >= 60) { seconds = 0; if(++minutes >= 60) { minutes = 0; hours++; } }`. Handle 24-hour wraparound if needed. Decrement works similarly in reverse. Include `toSeconds()` for total seconds since midnight. Display as HH:MM:SS. Test boundary cases like 23:59:59++ and 00:00:00--.

Exercise 29

Intermediate

Create a class `RingBuffer` with a position pointer. Overload increment/decrement operators to move the position circularly through the buffer.

Expected Implementation Details:

Store array, size, and current position. Increment: `position = (position + 1) % size`. Decrement: `position = (position - 1 + size) % size` to handle negative wrap. Include `read()` and `write()` at current position. Track full/empty state separately from position. Test circular movement in both directions, ensure proper wraparound.

Exercise 30

Difficult

Design a class `VersionNumber` with major.minor.patch format. Overload increment to follow semantic versioning rules (patch++ normally, but minor++ resets patch to 0).

Expected Implementation Details:

Default increment advances patch. Add methods `incrementMajor()`, `incrementMinor()` that reset lower components: major++ resets minor and patch to 0, minor++ resets patch to 0. Consider special increment that detects when to advance minor (e.g., patch >= 10). Include version comparison operators. Parse and format version strings. Test: 1.2.9++ = 1.2.10, not 1.3.0 unless special rules apply.

Category 5: Stream Operators (cout/cin)

Exercise 31

Easy

For the `Complex` class, overload `<<` to display as "a + bi" and `>>` to read complex numbers from input.

Expected Implementation Details:

Friend functions: `friend ostream& operator<<(ostream& os, const Complex& c)`. Handle negative imaginary: "3 - 2i" not "3 + -2i". Special cases: pure real "5", pure imaginary "2i", zero "0". For input, accept formats: "a+bi", "a-bi", "a", "bi". Parse carefully, handle spaces. Set failbit on invalid input. Test various formats and edge cases.

Exercise 32

Easy

Create a class `Book` with title, author, and ISBN. Overload `<<` for formatted output and `>>` for reading book details.

Expected Implementation Details:

Output format: "\"Title\" by Author (ISBN: 123-456)". For input, read line by line: first line title, second author, third ISBN. Or use delimited format: "Title|Author|ISBN". Handle titles/authors with spaces. Validate ISBN format if needed. Use `getline()` for strings with spaces. Return stream reference for chaining. Test with various book data.

Exercise 33

Intermediate

Design a class `Matrix`. Overload `<<` to display in row/column format and `>>` to read matrix elements with size specification.

Expected Implementation Details:

Output with aligned columns: use `setw()`. Format like: "[1 2 3]\n[4 5 6]" or with brackets for each row. Input: first read dimensions (rows cols), then read elements. Handle different matrix sizes dynamically. Include error checking for dimension mismatches. Option: support both space and comma-separated values. Test formatting with different sized elements.

Exercise 34

Easy

Implement a class `PhoneNumber`. Overload stream operators to handle various formats: +91-9876543210, (987) 654-3210, etc.

Expected Implementation Details:

Output in standard format: "+CC-AAAA-NNNNNNN". For input, accept multiple formats using regex or manual parsing. Extract country code, area code, number. Handle optional '+' prefix, various separators (-, space, .), parentheses. Validate component lengths. Set stream failbit for invalid formats. Test with international and domestic formats.

Exercise 35

Intermediate

Create a class `ChessBoard`. Overload `<<` to display the board with pieces using Unicode symbols, and `>>` to read moves in algebraic notation.

Expected Implementation Details:

Use Unicode chess symbols: ♔♚♜♞♝♟♙♛♡♣♤♢♣♞♟♙♛♡♣♤♢♣♞♟♙♛♡♣♤♢♣♞♟ for white, ♚♜♞♝♟♙♛♡♣♤♢♣♞♟♙♛♡♣♤♢♣♞♟ for black. Display with board coordinates (a-h, 1-8). Alternate square colors using background. For input, parse algebraic notation: "e2-e4", "Nf3", "O-O" (castling). Update board state based on moves. Include move validation. Handle special moves: en passant, promotion. Test complete game notation.

Exercise 36

Difficult

Design a class `XMLElement`. Overload `<<` to output valid XML with proper indentation, and `>>` to parse XML input with error handling.

Expected Implementation Details:

Output with proper nesting: "<tag attr=\"value\">content</tag>". Handle self-closing tags, attributes, child elements. Use recursion for nested elements with indentation. For input, implement basic XML parser: handle opening tags, attributes, content, closing tags. Check matching tags. Store as tree structure. Escape special characters: < > & " ' . Test nested structures and malformed XML.

Category 6: new/delete Operators

Exercise 37

Intermediate

Create a class `MemoryPool` that overloads `new` and `delete` operators to allocate objects from a pre-allocated memory pool.

Expected Implementation Details:

Static `char buffer[POOL_SIZE]` and free list. `void* operator new(size_t size)` returns next free block from pool. Maintain free list as linked list within unused blocks. `operator delete(void* p)` returns block to free list. Handle pool exhaustion by falling back to global `new` or throwing. Include static initialization of pool. Test allocation patterns and pool reuse.

Exercise 38

Difficult

Design a class `TrackedObject` that overloads `new` and `delete` to maintain allocation statistics (count, total memory, peak usage).

Expected Implementation Details:

Static members: allocation count, current bytes, peak bytes. In `new`: increment count, add size to current, update peak if needed, allocate with global `::new`. In `delete`: decrement count, subtract size (need to store size with allocation). Create allocation header struct before actual object. Include static methods to report statistics. Handle array `new[]` and `delete[]` separately.

Exercise 39

Difficult

Implement a class `AlignedData` that overloads `new` to ensure memory is aligned to specified boundaries (e.g., 16-byte alignment for SIMD operations).

Expected Implementation Details:

Calculate aligned address: $(\text{address} + \text{alignment} - 1) \& \sim(\text{alignment} - 1)$. Allocate extra space for alignment adjustment and store original pointer. `operator new(size_t size, size_t alignment)` for custom alignment. Store offset to original allocation before object. In delete, calculate original pointer from offset and free it. Test alignment with various boundaries: 8, 16, 32 bytes.

Exercise 40

Difficult

Create a class `DebugObject` that overloads `new` and `delete` to detect memory leaks, double deletes, and buffer overruns using guard bytes.

Expected Implementation Details:

Add guard bytes (e.g., 0xDEADBEEF) before and after allocated memory. In new: allocate extra space for guards, set guards, maintain allocation list. In delete: check guards for corruption, check if pointer is in allocation list (detect double delete), remove from list. At program end, report any remaining allocations as leaks. Include file/line info using placement new. Test various error conditions.